INTERACTION SPACES: NOTES FOR SOFTWARE DESIGN

1. Interaction spaces

We refer to [2, Sec. 2] for the most updated definition of IS.

We have to distinguish when an interacting entity (e.g. "a rectangle") or an interaction (e.g. "is internal") belongs to an IS or another one. E.g. it can belong to one training left grid in ARC-AGI 1, or to the right grid, or being the mental representation in another IS. Learning to think at them as different interacting entities helps in the construction of CEF, which maps these entities among each other. For this reason, it seems simpler to use UML's aggregation (empty rhombus, call directly with the entity constructor, e.g. `InteractingEntity`) and not composition (black rhombus, call with parameters and instantiation internal to `InteractionSpace`). For example, we can first construct an interacting entity (e.g. a rectangle) and then associate it to two different IS. For this motivation, we will introduce an attribute that specifies which IS this entity is associated with. We can therefore have e.g. two interacting entity $e_1 \in E_1$, $e_2 \in E_2$ having the same values of state $x_{e_1}(t) = x_{e_2}(t)$, but aggregated to two different IS (therefore, they are different `InteractingEntity` objects because the associated IS are different: `e_1.IS ! = e_2.IS` (see below).

In the class `InteractionSpace` we surely set as attributes all the variables and quantities that are parameters for the considered IS. For example, the time t of all the time-depending functions, such as $t_i^{\mathrm{s}}(t)$, $t_i^{\mathrm{o}}(t)$, $\mathrm{ac}_i^r(t)$ has to be implemented in the code with asynchronous or synchronous steps depending on the model (see the dynamics of an IS in [1, Sec. 2.8]).

In the following notes, sometimes we use the customary `self` and sometimes another more meaningful name.

In the following, Type Hints are used only *informally*.

1.1. Attributes and methods of `InteractionSpace`.

(i) Attributes:

(i) `id: str`

(ii) `t_st: float`
- t_{st}

(iii) `t_end: float`
- t_{end}

(iv) `t: float`
- Corresponds to the last value of time $t \in [t_{\mathrm{st}}, t_{\mathrm{end}}]$ in the algorithm explained in [1, Sec. 2.8], i.e. it takes as values of the times when anything occurs in the implemented IS.

(v) `E: dict[id: str, e:InteractingEntity]`
- This corresponds to the set E of all the interacting entities of the IS.
- We first have to recognize an interacting entity and collect the values of its attributes. Using the class `InteractionEntity`, we instantiate `e`, then we add this entity to the corresponding IS, with `IS.E[id_of_e] = e`.
- Keys and values of `E` are `(e.id: e)`, for each `InteractingEntity e`, therefore: `self.E[e.id] == e` iff $e \in E$.

(vi) `I: dict[id: str, i:Interaction]`
- This corresponds to the set I of all the interactions of the IS.
- We first have to recognize an interaction and collect the values of its attributes. Using the class `Interaction`, we instantiate `i`, then we add this entity to the corresponding IS, with `IS.I[id_of_i] = i`.
- Keys and values of `I` are `(i.id: i)`, for each `Interaction i`, therefore: `self.I[i.id] == i` iff $i \in I$.

(vii) `Delta: float`
- `self.Delta` corresponds to Δ (in the evolution equation).
- Open problem: it would be computationally more efficient to have Δ_p, i.e. depending on the patient p, in the evolution equation (see [2, Def. 8]). However, the corresponding algorithm for the dynamics of an IS [1, Sec. 2.8] would probably be more involved. Can you understand this algorithm? Start thinking at only two patients having different Δ_p, e.g. two moving balls having an elastic collision.

(viii) `is_static: bool`
- `self.is_static == True` if the IS is declared as static.
- Formally, an IS is static if
  - All the states $x_e(t)$ are constant in $t \in [t_{\text{st}}, t_{\text{end}}]$ and all $e \in E$;
  - For all interactions i, we have $t_i^{\text{s}}(t_{\text{st}}) = t_i^{\text{o}}(t_{\text{st}}) = t_{\text{st}}$ (i.e. each interaction i starts at t_{st}) and $t_i^{\text{s}}(t) = t_i^{\text{o}}(t) = +\infty$ (i.e. i never restart again) for all $t > t_{\text{st}}$.

  From this, it follows that $t^1(t_{\text{st}}) = t_{\text{st}}$, $t^1(t) = +\infty$ for all $t > t_{\text{st}}$ and $I_p(t) = \{i \in I \mid \text{pa}(i) = p\}$. Therefore, the EE can be applied only for $t = t_{\text{st}}$ and for all $t^1(t_{\text{st}}) = t_{\text{st}} \leq s \leq t_{\text{st}} + \Delta$, we must have $x_p = f_p(\omega, s, \text{n}_p\, x_s)$, i.e. the right-hand-side of the EE does not depend neither on ω nor on s.

(ii) Methods:

(i) `t_first_arrival(t: float) -> float`
- `IS.t_first_arrival(t) = min((i.ta[t] for i in self.I.values() if i.ts[t] == t), default = float('inf'))`
- it corresponds to: $t^1(t) = \min\{t_i^{\text{a}}(t) \mid i \in I,\ t_i^{\text{s}}(t) = t\}$, see [2, (2.4)].
- For static IS, the previous implementation of `IS.t_first_arrival` should give the correct results, i.e. `IS.t_first_arrival(IS.t_st) == IS.t_st` and `IS.t_first_arrival(IS.t) == float(''inf'')` if `IS.t > IS.t_st` if we implement correctly the starting and arrival times of the interactions of a static IS.

(ii) `run() -> dict`
- Set all the needed initial conditions: proper state variables $x_e(t_{\text{st}})$, activations $\text{ac}_i^e(t_{\text{st}})$, goods $\gamma_i(t_{\text{st}})$, $t_i^{\text{s}}(t_{\text{st}})$, $t_i^{\text{o}}(t_{\text{st}})$, $t_i^{\text{a}}(t_{\text{st}})$, $\mathcal{N}_i(t_{\text{st}})$.
- Follows the dynamics explained in [1, Sec. 2.8].

(iii) `execution_log(path: str) -> None`
- Save in the file `path` all the interesting variables of the IS.

(iv) `load_system_state(path: str) -> None`
- Load from the file `path` all the IS variables (e.g. load an entire initial condition)

2. Interacting entities

Every interacting entity $e \in E$ is an object in the class `InteractingEntity`. These are classified (as we prefer) into subclasses: in the design phase of the problem (e.g. ARC-AGI 1), we decide what are the subclasses and what are the sub-subclasses (e.g. `Rectangle` is a subclass of `ClosedLine`), etc.

Each attribute, from the mathematical point of view, is a function $t \mapsto x_e(t)$. We can implement this using two methods of the form `get_state(t: float)` and `set_state(t: float)` like in Guntur's UML diagram "PlantUML IS". On the other hand, it would be more natural to see $x_e(t)$ as the value of an attribute of e at time t. We can therefore implement them as dictionaries of the form `e.x[t]`. In this way, we obtain a notation more similar to the mathematical one and having read and write properties. Moreover, in the entire dictionary `e.x` we have all the pairs (key, value) of the form `t: e.x[t]`, i.e. each time event and the related value of the state (i.e. the history of the state $x_e(t)$, or the graph of the function $t \mapsto x_e(t)$). This can be useful to study the dynamics of e.

2.1. About the implementation of the state space.

In a first implementation, we can avoid to implement the state space. In fact, the state space is useful as a "stand alone object", e.g. in the following case:

(i) We model all the possible states of all the interacting entities with the space $\bar{M}$ (see [2, Sec. 3]).

(ii) We wan to study the GEP in $\bar{M}$, i.e. what are good possible global configurations of our IS.

This "static" approach is different from a dynamical/runtime calculation of the GEP at time t, where we consider only the state $x_e(t)$ of each interacting entity e (in order to evaluate costs/unification) at time t, and the goods $\gamma_i(t)$ of each interaction i at the same time (in order to evaluate entropy/diversification). Up to now, we are more interested to the dynamical approach.

On the contrary, the space of all the resources $R_i \ni \gamma_i(t)$ (for each interaction) can be useful to consider phenomena such as "depletion of resources" or "for normalizing the spaces of resources in order to obtain a meaningful probability for the diversification of the GEP", see [2, Sec. 8.3.2].

For these motivations, and only in case we are interested in the static approach, among the attributes of each entity e, we can simply implement the proper state space S_e as a tuple of `types` of the form (e.g.)

```
(numpy.ndarray, str, numpy.ndarray, int)
```

which mathematically correspond to $\mathbb{R}^{d_1} \times \text{Str} \times \mathbb{R}^{d_2} \times \mathbb{Z}$.

In case we need to consider e.g. a subspace of $\mathbb{R}^d$, e.g. an interval, we can implement the corresponding state space as a pair (2-tuple) given by a `type` (as above) and a boolean condition on that type `T`, i.e. a function `f(x:`

`T) -> bool`. This pair represents the set

$$\{x \in \texttt{T} \mid f(x) = \texttt{True}\}, \tag{2.1}$$

where $x \in \texttt{T}$ means `isinstance(x, T) == True`. For example, `f = lambda x: 0 <= x <= 1`.

Finally, in [2, Def. 6], the neighborhood $\mathcal{N}_p(t)$ of a patient $p \in E$ is needed only as a domain of the neighborhood state function $\mathrm{n}_p\, x_s : (\tau, \varepsilon) \in \mathcal{N}_p(t),\ \tau \leq s \mapsto x_\varepsilon(\tau) \in S_\varepsilon$. For this reason, we do not implement $\mathcal{N}_p(t)$ as an attribute of the subclass `Patient(InteractingEntity)`.

2.2. Attributes and methods of `InteractingEntity`.

(i) Attributes:
- (i) `id: str`
- (ii) `IS: InteractionSpace`
  - (i) The IS the interacting entity is associated with.
- (iii) `proper_state_dim: int`
- (iv) `states_description: list[str]`
  - for each component $x_e(t)^j$ a string `e.states_description[j]` describing the meaning of the j-th state. This description does not depend on t.
- (v) `ac: dict[tuple[i: Interaction, t: float], v: float] | dict[i: Interaction, v: float]`
  - If the associated IS is not static (i.e. `self.IS.is_static == False`), then `e.ac[i, t]` corresponds to $\mathrm{ac}_i^e(t) \in [0, 1]$.
  - If the associated IS is static (i.e. `self.IS.is_static == True`), then we can simply use `e.ac[i]` which have to correspond to the constant value $\mathrm{ac}_i^e(t_{\mathrm{st}}) \in [0, 1]$.
- (vi) `x: dict[t: float, v: numpy.ndarray] | numpy.ndarray`
  - If the associated IS is not static (i.e. `self.IS.is_static == False`), then `e.x[t]` corresponds to $x_e(t) \in S_e$ (the proper state space).
  - If the associated IS is static (i.e. `self.IS.is_static == True`), then we can simply use `e.x` which have to correspond to the constant value `(numpy.ndarray)` $x_e(t_{\mathrm{st}})$.
  - We use this notation only for proper state variables, but we have to recall that also activation and goods are dynamical state variables. For example, in general, both are included in the evolution equation (EE).
  - Assume that we need to use the speed features of Numpy for a numerical array `e.x[t]`. However, we also need other state variables such as `str` or more generic objects. In that case, it could be convenient to consider only the calculations with the `numpy.ndarray` contained in `e.x[t]`
- (vii) methods:

2.3. `Propagator` as subclass of `InteractingEntity`.

We have to recall that propagators are interacting entities, and not only attributes of interactions carrying the goods sent by agents to patients. This means that also propagators r have a proper state $x_r(t)$ and activations $\mathrm{ac}_i^r(t)$. For example, if $\mathrm{ac}_i^r(t_i^{\mathrm{a}}(t)) = 0$, then the interaction i cannot start (see axiom (CE) in [2, Def. 5]).

(i) `Propagator(InteractingEntity)`
- (i) attributes:
  - (i) `interactions: list[Interaction]`
    - The list of all the interactions i such that $\mathrm{pr}(i) = r$.
  - (ii) `type: str`
    - `r.type = r.id`, so that it is only another name.
    - It corresponds with the old "type of an interaction", and now equals the string `id` of the interaction. Therefore, if $\mathrm{pr}(i) = r$, then `r.type == i.id`
    - If we are thinking at two interactions intuitively having "the same entities", but actually of different "types", this means that they must have different propagators because r_1`.type == ''doing this'' == i_1.id` and and r_2`.type == ''doing that'' == i_2.id`.
  - (iii) `gamma: dict[tuple[i: Interaction, t: float], v: Any] | dict[i: Interaction, v: Any]`
    - If the associated IS is not static (i.e. `self.IS.is_static == False`), then `r.gamma[i, t]` corresponds to $\gamma_i(t) \in R_i$, where $\mathrm{pr}(i) = r$, i.e. if: `i in r.interactions == True`
    - If the associated IS is static (i.e. `self.IS.is_static == True`), then we can simply use `r.gamma[i]` which corresponds to the constant value $\gamma_i(t_{\mathrm{st}}) \in R_i$.
    - We also have: `type(r.gamma[i, t]) == i.resources_type` (see below Sec. 4.1).

(ii) methods:

2.4. `Patient` as subclass of `InteractingEntity`.

(i) `Patient(InteractingEntity)`
  (i) attributes:
    (i) `interactions: list[Interaction]`
    - The list of all the interactions i such that $\mathrm{pa}(i) = p$.

    (ii) `I: dict[t: float, I_list: list[Interaction]] | list[Interaction]`
    - If the associated IS is not static (i.e. `self.IS.is_static == False`), then `p.I[t]` is a list having the same elements of $I_p(t)$ (see [2, Sec. 2.4]). In other words, `set(p.I[t])` corresponds to $I_p(t)$.
    - If the associated IS is static (i.e. `self.IS.is_static == True`), we can use `p.I`, which corresponds to the constant value $I_p(t_{\mathrm{st}}) = \{i \in I \mid \mathrm{pa}(i) = p\}$, which equals `set(p.interactions)`
    - It can be implemented as (see `IS.py`):

```
self.I: dict[float, list[Interaction]] | list[Interaction]
        if self.IS.is_static == False:
            t1 = self.IS.t_first_arrival(self.IS.t)
            self.I[self.IS.t] = [i for i in self.interactions if
                                 t1 <= i.ta[self.IS.t] <= t1 +
   self.IS.Delta]
        else:
            self.I = self.interactions
```

    - It seems that we need to use `p.I[t]` only to test if it is non empty for the implementation of the evolution equation, that is in a code of the form: `if p.I[t]: EE`

    (iii) `Omega_type: str | None = None`
    - `p.Omega_type = ''discrete''` if $(\Omega_p, \mathcal{F}_p, P_p)$ is a discrete probability space.
    - `p.Omega_type = ''continuous''` if $(\Omega_p, \mathcal{F}_p, P_p)$ is a continuous probability space.
    - In case we need to implement more general probability spaces, we have to use generalized functions as densities. Moreover, up to now, continuous probability spaces are only on a 1-dim. interval.
    - If the IS is static, we omit the variable (`Omega_type = None`).

    (iv) `Omega_discr_set: set | None = None`
    - If `Omega_type = ''continuous''`, then `Omega_discr_set = None`
    - Otherwise, if $\Omega_p = \{\omega_1, \ldots, \omega_K\}$, then `Omega_discr_set = {omega_1, ..., omega_K}`
    - If the IS is static, we omit the variable.

    (v) `Omega_int_min: float | None = None`
    - If `Omega_type = ''discrete''`, then `Omega_int_min = None`
    - Otherwise, if $\Omega_p = [a, b]$, then `p.Omega_int_min = a`. Recall that we can take `a = float(''-inf'')` or `b = float(''inf'')`.
    - If the IS is static, we omit the variable.

    (vi) `Omega_int_max: float | None = None`
    - If `Omega_type = ''discrete''`, then `Omega_int_max = None`
    - Otherwise, if $\Omega_p = [a, b]$, then `p.Omega_int_max = b`
    - If the IS is static, we omit the variable.

    (vii) `P_density: FunctionType | None = None`
    - If `Omega_type = ''discrete''`, then `P_density(omega in Omega_discr_set) -> float`
    - $P_p(\{\omega\})$ corresponds to `p.P_density(omega)`
    - If `Omega_type = ''continuous''`, then `P_density(Omega_int_min <= omega <= Omega_int_max) -> float`
    - If $\Omega_p = [a, b]$, the probability P_p is distributed with density `p.P_density(omega)`
    - If the IS is static, we omit the variable.

    (viii) `f: FunctionType | None = None`

- The function `p.f` is implemented in the code and run in the package `Evolution System`, where we implement the evolution equation. Therefore, it is a function of the form `p.f(omega, s, p.nx(t,s))`.
- In `Evolution System`, we have to implement `p.x(s) = p.f(omega, s, p.nx(t,s))` for all `IS.t_first_arrival(t) <= s <= IS.t_first_arrival(t) + IS.Delta` (for a certain number of `s` if `IS.Delta > 0`).
- This implementation has to be realized after extracting an elementary events `omega` using the previous probability space $(\Omega_p, \mathcal{F}_p, P_p)$ and only if:
  - `IS.t_first_arrival(t) < inf`
  - `if p.I[t]:` i.e. if `p.I[t]` is not empty.
- If the IS is static, we omit the variable.

(ii) methods:

(i) `nx(t: float, s: float) -> dict[tuple[tau: float, eps: InteractingEntity], v: numpy.ndarray]`

- `p.nx(t, s)` corresponds to the function $(\tau, \varepsilon) \mapsto \mathrm{n}_p\, x(\tau, \varepsilon) = x_\varepsilon(\tau) \in S_\varepsilon$, see [2, Def. 6] (which actually depends also on t because its domain $\mathcal{N}_p(t)$ depends on t).
- `p.nx(t, s)` is the dict with keys `(tau, eps)` and values `v = eps.x[tau]` constructed as follows:
  - Consider all the interactions having p as patient: `i in p.interactions`
  - For these interactions i, consider all the past arrival times $t' = t_i^{\mathrm{a}}(t') \leq t$ of i: `t_prime in i.arrival_times() & t_prime <= IS.t`
  - In the code above, `IS` is the IS where we are instantiating the `Interaction` i (composition).
  - Consider all the interacting entities $\varepsilon \in \mathcal{N}_i(t')$ at these times t': `eps in i.N(t_prime)`
  - `tau` are all the events times of $x_\varepsilon(-)$, i.e. times when the state of ε changes: `eps.x.keys()` and such that `tau <= s`
  - Pairs (key: value) are: `(tau, eps): eps.x[tau]`
  - In this way, we have: `p.nx(t)[tau, eps] == eps.x[tau]` for the correct (τ, ε) as in [2, (2.5)].
- If the IS is static, we simply do not use this method (the attribute `p.f == None` and the EE is useless).

3. Discr./cont. time events

Sets T of discrete or continuous time events (see [2, Def. 3]) are what is actually generated, depending on the model we are considering, using the general algorithm described in [1, Sec. 3.5.1]. Intuitively, they are all the times when something occurs: an interaction starts, a propagator arrives, the state of an interacting entity changes, the neighborhood of an interaction changes, etc (all these time events are actually all the values taken by `IS.t`). Each clock function τ is defined by one and only one of these set of time events because $T = \tau([t_{\mathrm{st}}, t_{\mathrm{end}}])$. We therefore introduce these T as a new class of objects and the corresponding clock function as one of their methods.

We recall that the unions in a set of discr./cont. time events $T = \bigcup_{j\in N}\{t_j\} \cup \bigcup_{k\in M}[t_k^1, t_k^2]$ must all be disjoint, so that t_j are all different, $t_k^1 < t_k^2$, no t_j lies in some $[t_k^1, t_k^2]$, and all the intervals $[t_k^1, t_k^2]$ are disjoint. The (stochastic) generation of T naturally respect these constraints, see [1, Sec. 3.5.1].

On the other hand, $t \mapsto t_i^{\mathrm{a}}$ is *not* a clock function ([1, Sec. 3.4]). However, we (stochastically) generate $t_i^{\mathrm{a}}(t)$ using the algorithm [1, Sec. 3.5.1] starting from a discrete set of times $T_i^{\mathrm{a}} = \bigcup_{j\in N_{\mathrm{a}}}\{t_j^{\mathrm{a}}\}$. We can therefore include this T_i^{a} as a particular case of a set of discr./cont. time events.

(i) `TimeEvents`

(i) attributes:

(i) `IS: InteractionSpace`

- The IS the set of discr./cont. time events is associated with.

(ii) `discrete: list[float]`

- `T.discrete[j]` corresponds to t_j if $T = \bigcup_{j\in N}\{t_j\} \cup \bigcup_{k\in M}[t_k^1, t_k^2]$. The list `T.discrete` is sorted and all time events are different, i.e. $t_j < t_{j+1}$.

(iii) `continuous: numpy.ndarray | None`

- If this attribute `is not None`, then:
  - `T.continuous[k, 0]` corresponds to t_k^1.
  - `T.continuous[k, 1]` corresponds to t_k^2.
  - The array represent disjoint intervals, i.e. $t_k^1 < t_k^2 < t_{k+1}^1 < t_{k+1}^2$.

- If this attribute `is None`, then this is the case of the set of time events that generates $t \mapsto t_i^{\mathrm{a}}(t)$.
- If this attribute `is not None`, it means that we have a set of discr./cont. time events that can generate a clock function, but having no continuous time events.

(iv) `infty: bool`
- `T.infty == True` iff $+\infty \in T$ (in this case $T = \bigcup_{j\in N}\{t_j\} \cup \bigcup_{k\in M}[t_k^1, t_k^2] \cup \{+\infty\}$)

(ii) methods:

(i) `clock(t: float) -> float`
- It is the clock function generated by the set T, i.e. $\tau(t) = \min\{s \geq t \mid s \in T\}$. Recall that $\min(\emptyset) = +\infty$.
- It can be implemented as in `IS.py`

In case of static IS, we do not use this class because it is trivial.

4. Interactions

Firstly think at interactions as *formal* $n+2$*-tuples* $i = (a_1, \ldots, a_n, r, p)$ and after as *dynamical entities* characterized by t_i^{s}, t_i^{o}, ac_i^r... Interactions $i = (a_1, \ldots, a_n, r, p)$, or $i : a_1, \ldots, a_n \xrightarrow{r} p$, are objects in the class `Interaction`.

4.1. Attributes and methods of `Interaction`.

(i) `Interaction`:

(i) attributes:

(i) `id: str`
- This `self.id` and the old "type of an interaction" are the same concept.

(ii) `IS: InteractionSpace`
- The IS the interaction is associated with.

(iii) `agents: tuple of InteractingEntity`
- This is the n-tuple of agents $a_1, \ldots, a_n$.

(iv) `pr: Propagator`
- Propagator r.
- At each instantiation of an interaction `i`, we set the attribute `i.pr.type = self.id`

(v) `pa: Patient`
- Patient p.

(vi) `resources_type: Any | None = None`
- If this attribute is `None`, the following is not implemented (the same below).
- `It` can be any type: `float | int | str | FunctionType` ... (or any user define type).

(vii) `resource_space_dim: int | None = None`
- If $R_i = \mathbb{R}^d$, then `resource_space_dim = n`, otherwise `resource_space_dim = -1`.

(viii) `R: tuple(Any, types.FunctionType) | None = None`
- We must have `T == self.resources_type`
- As we explained above in (2.1), if we have `i.R == (T, f)`, then $R_i = \{x \in \mathtt{T} \mid \mathtt{f}(x) == \mathtt{True}\}$
- Therefore, `f` is of the form: `f(x: T) -> bool`

(ix) `min_resource: float | None = None`
- simple case where $R_i =$ `[i.min_resource, i.max_resource]` is an interval of $\mathbb{R}$

(x) `max_resource: float | None = None`

(xi) `T_s: TimeEvents | None = None`
- `i.T_s` is the set of discr./cont. time events that generates the clock function t_i^{s}.
- Recall that this set T_i^{s} increases with t: see the dynamics of an IS in [1, Sec. 3.5.1].
- If the IS is static, we can omit this parameter.

(xii) `T_o: TimeEvents | None = None`
- `i.T_o` is the set of discr./cont. time events that generates the clock function t_i^{o}.
- Recall that this set T_i^{o} increases with t: see the dynamics of an IS in [1, Sec. 2.8].
- If the IS is static, we can omit this parameter.

(xiii) `T_a: TimeEvents | None = None`
- `i.T_a` is the set of discrete time events that generates the function t_i^{a}.
- Recall that this set T_i^{a} increases with t: see the dynamics of an IS in [1, Sec. 2.8].
- If the IS is static, we can omit this parameter.

(xiv) `ts: dict[t: float = IS.t, t_start: float]`

- `i.ts[t]` corresponds to $t_i^{\mathrm{s}}(t)$.
- `i.ts[t] = i.T_s.clock(t)`, where `t = i.IS.t` is the current IS time.

(xv) `to: dict[t: float = IS.t, t_ongoing: float]`
- `i.to[t]` corresponds to $t_i^{\mathrm{o}}(t)$.
- `i.to[t] = i.T_o.clock(t)`

(xvi) `ta: dict[t: float = IS.t, t_arrival: float]`
- See `IS.py` for a possible implementation. The idea is to set `i.ta[t] = i.T_a.discrete[j]` if `i.T_a.discrete[j] <= IS.t < i.T_a.discrete[j+1]` which corresponds to $t_i^{\mathrm{a}}(t) = t_j^{\mathrm{a}}$ if $t_j^{\mathrm{a}} \leq t < t_{j+1}^{\mathrm{a}}$ and $T_i^{\mathrm{a}} = \bigcup_{j \in N_{\mathrm{a}}} \{t_j^{\mathrm{a}}\}$.

(xvii) `N: dict[t: float = IS.t, N_list: list[InteractingEntity]] | list[InteractingEntity]`
- If the associated IS is not static (i.e. `self.IS.is_static == False`), then `i.N[t]` corresponds to $\mathcal{N}_i(t)$ but it is implemented as a list and not as a set (saying that the neighborhood is a set in an inheritance from cellular automata theory, but it is not completely correct if the evolution function depends on some order of the entities in this neighborhood (e.g. because the CA is on a spatial grid). Using a list, we can possibly use a meaningful indexing depending on the model of the interacting entities inside $\mathcal{N}_i(t)$).
- If the associated IS is static (i.e. `self.IS.is_static == True`), then we can simply use `i.N` which have to correspond to the constant value $\mathcal{N}_i(t_{\mathrm{st}}) \subseteq E$.

(ii) methods:

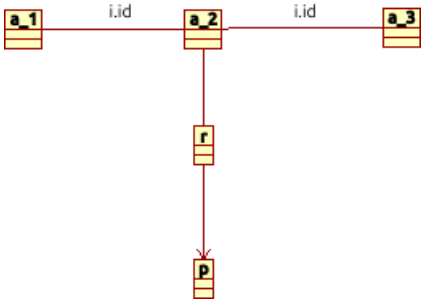
(i) `arity() -> int`
- `arity(i) = len(self.agents)` is the number n of agents $a_1, \ldots, a_n$.

(ii) `ag() -> tuple`
- `ag(i) = self.agents`. This method simply implement the mathematical notation $\mathrm{ag}(i) = (a_1, \ldots, a_n)$

(iii) `a(j: int) -> InteractingEntity`
- `a(i, j) = self.agents[j]` corresponds to a_j in the interaction i

(iv) `gamma(t: float | None = None) -> Any`
- If the IS is not static: `gamma(i, t) = pr(i).gamma[i, t] == i.pr.gamma[i, t]` this is $\gamma_i(t)$, i.e. the goods of the interaction i.
- If the IS is static: `gamma(i) = pr(i).gamma[i] == i.pr.gamma[i]`

(v) `UML() -> None`
- is the UML (png figure) of the interaction:



- An external function `khx(list_active_interactions, IS.t)` will take as input the list of all the active interactions at time t

(iii) `nx(t: float) -> dict[tuple[tau: float, eps: InteractingEntity], v: numpy.ndarray]`
- `i.nx(t)` corresponds to the function $(\tau, \varepsilon) \mapsto \mathrm{n}_i\, x(\tau, \varepsilon) = x_\varepsilon(\tau) \in S_\varepsilon$, see [1, (3.4)].
- `i.nx(t)` is the dict with keys and values constructed as follows:
  - Consider all the past arrival times $t' = t_i^{\mathrm{a}}(t') \leq t$ of i: `t_prime in i.arrival_times() & t_prime <= IS.t`
  - In the code above, `IS` is the IS where we are instantiating this `Interaction` (composition).
  - Consider all interacting entities $\varepsilon \in \mathcal{N}_i(t')$ at these times t': `eps in i.N(t_prime)`
  - `tau` are all the events times of $x_\varepsilon(-)$, i.e. times when the state of ε changes: `eps.x.keys()`
  - Pairs (key: value) are: `(tau, eps): eps.x[tau]`
  - In this way, we have: `i.nx(t)[tau, eps] == eps.x[tau]` for the correct (τ, ε) as in [1, (3.4)].
- The formula [1, (3.4)] is less formal because the dependence on t is implicit.
- The (stochastic) generation of clock functions (see [1, Sec. 3.5.1]) in general depends on this neighborhood $\mathrm{n}_i\, x = \mathrm{n}_i\, x(t)$.

- If the IS is static, we simply do not use this method.

(iv) `starting_times() -> list[float]`
- The ordered list of all the starting times.
- `i.starting_times = sorted(i.T_s.discrete + i.T_s.continuous.flatten())`

(v) `arrival_times() -> list[float]`
- The list of all arrival times.
- `i.arrival_times = T_a.discrete`

(vi) `ongoing_times() -> list[float]`
- The ordered list of all the ongoing times.
- `i.ongoing_times = sorted(i.T_o.discrete + i.T_o.continuous.flatten())`

References

[1] Giordano P., Interaction spaces I: Towards a unified mathematical theory of complex systems. See http://www.mat.univie.ac.at/~giordap7/IS.pdf

[2] Giordano P., Interaction spaces II: A mathematical definition of complex adaptive systems as interaction spaces. See http://www.mat.univie.ac.at/~giordap7/IS_cas.pdf